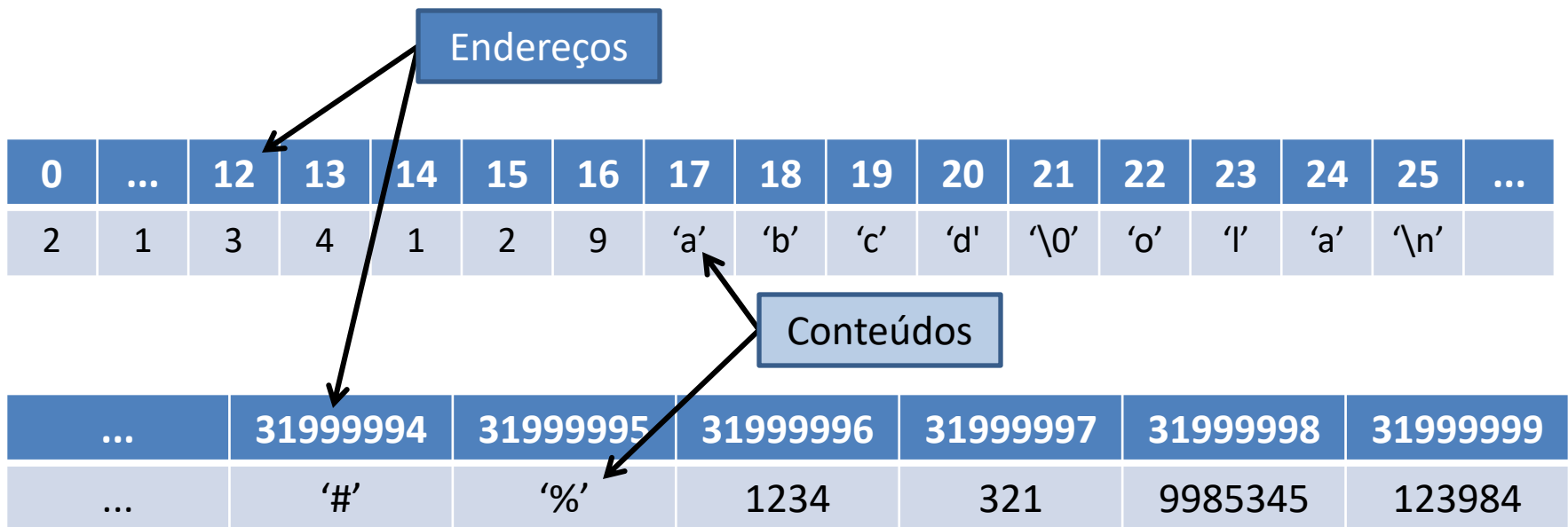


Aula 05: Ponteiros

Variáveis e Endereços

- Memória: sequência contínua de posições endereçáveis
- Cada posição armazena informação: desde a posição zero até a posição máxima de memória.



Variáveis e Endereços

- Variáveis: associadas a endereços de memória

```
int x = 2;  
char str1[] = "abcd";  
char str2[] = "olá!";
```

Obs.: inteiros ocupam 4 bytes enquanto cada caractere ocupa 1 byte

0	...	12	13	14	15	16	17	18	19	20	21	22	23	24	25	...
2	1	2				'a'	'b'	'c'	'd'	'\0'	'o'	'l'	'a'	'!'	'\0'	...
		x				str1					str2					

...	31999994	31999995	31999996	31999997	31999998	31999999
...	'#'	'%'	1234	321	9985345	123984

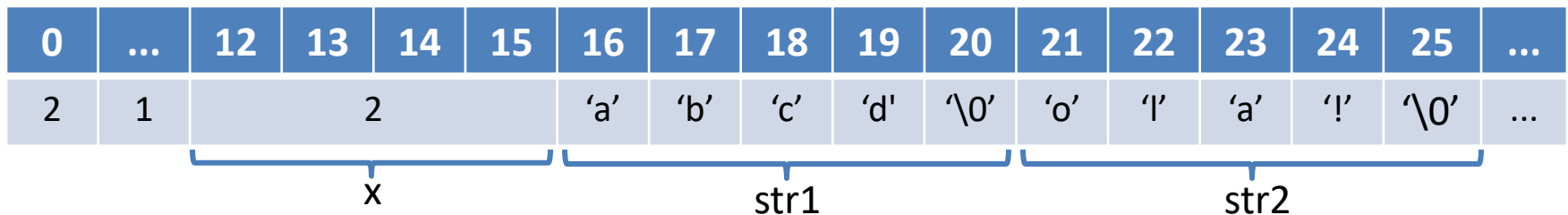
Lixo de memória que sobrou de execuções de programas anteriores

Variáveis e Endereços

- Conteúdo de cada variável está armazenado em um endereço de memória e possui um tamanho

```
int x = 2;  
char str1[] = "abcd";  
char str2[] = "olá!";
```

- Memória:



Variável	Endereço	Tamanho	Conteúdo
x	?	?	?
str1	?	?	?
str2	?	?	?

Endereço, Tamanho, e Conteúdo

- A notação utilizada para se obter:
 - Conteúdo de uma variável: `x` (ou `x[i]` para vetores)
 - Endereço: `&x` (ou `x` para vetores)
 - Tamanho: `sizeof(x)` (ou `sizeof(tipo)` para saber o tamanho de um tipo)

Obs.: para vetores, o nome da variável já indica também seu endereço inicial.

```
#include <stdio.h>
#include <string.h>
int main()
{
    int x = 2;
    char str1[] = "abcd";
    char str2[] = "ola!";

    printf("x:\n Endereço: %d\n Tamanho: %d\n Conteúdo: %d\n\n",
           &x, sizeof(x), x);
    printf("str1:\n Endereço: %d\n Tamanho: %d\n Conteúdo: %s\n\n",
           str1, sizeof(str1), str1);
    printf("str2:\n Endereço: %d\n Tamanho: %d\n Conteúdo: %s\n\n",
           str2, sizeof(str2), str2);
    printf("str2:\n Endereço: %d\n Tamanho: %d\n Conteúdo: %s\n\n",
           &str2[0], sizeof(char)*(strlen(str2)+1), str2);
    return 0;
}
```

```
x:
Endereco: 2293580
Tamanho: 4
Conteúdo: 2

str1:
Endereco: 2293575
Tamanho: 5
Conteúdo: abcd

str2:
Endereco: 2293570
Tamanho: 5
Conteúdo: ola!

str2:
Endereco: 2293570
Tamanho: 5
Conteúdo: ola!
```

Declarando Variáveis do Tipo Ponteiro em C

- Para declarar: notação '*':

int x:

- Declara uma variável que armazena um inteiro;

int *y:

- Declara uma variável que armazena o endereço de memória de um inteiro. Ou, como se diz: guarda um ponteiro para uma variável do tipo inteiro.

- Forma geral:

tipo *nomeDoPonteiro;

Operador &

- Operador unário que fornece o endereço de uma variável:
- Forma geral:

`&nomeDaVariavel`

```
int x;  
int *y;  
y = &x;
```

Faz o ponteiro y apontar para x

Obs.: Esse operador não pode ser utilizado com literais:

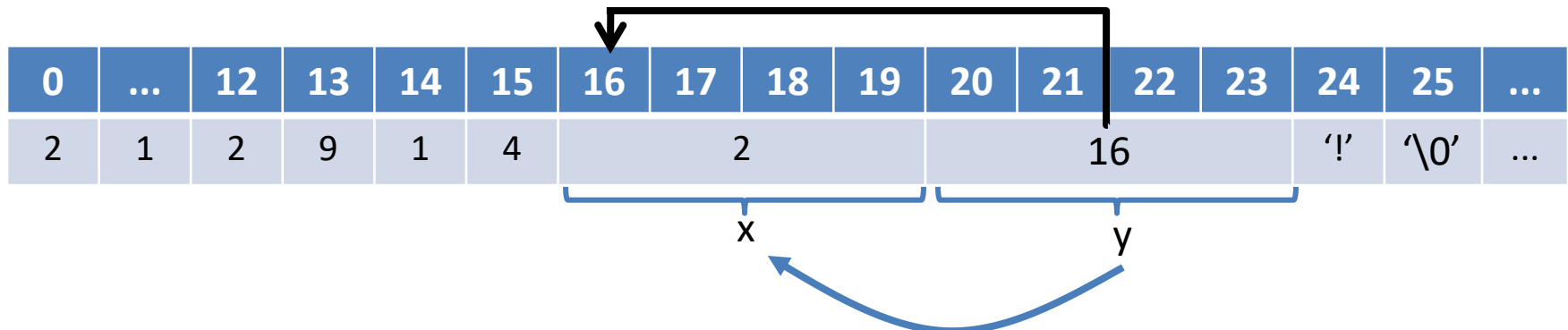
`y = &3;` //Errado!

Ponteiros

x:
Endereco: 16
Tamanho: 4
Conteudo: 2

y:
Endereco: 20
Tamanho: 4
Conteudo: 16
Conteudo apontado: 2

```
int main ()  
{  
    int x = 2;  
    int *y = &x;  
  
    printf("x:\n Endereco: %d\n Tamanho: %d\n Conteudo: %d\n\n",  
          &x, sizeof(x), x);  
    printf("y:\n Endereco: %d\n Tamanho: %d\n Conteudo: %d\n Conteudo apontado: %d\n\n",  
          &y, sizeof(y), y, *y);  
    return 0;  
}
```



- Dizemos que **y** aponta para **x**

Utilizando Ponteiros

```
int i, j;
```

```
int *ip;
```

```
i = 12;
```

```
ip = &i;
```

```
j = *ip;
```

```
*ip = 21;
```

A variável **ip** armazena um ponteiro para um inteiro

O endereço de **i** é armazenado em **ip**

O conteúdo da posição apontada por **ip** é armazenado em **j**

O conteúdo da posição apontada por **ip** passa a ser **21**

Utilizando Ponteiros

1)

```
int i,j ;
```

```
int *ip ;
```

ip	-	112
j	-	108
i	-	104

2)

```
i = 12 ;
```

ip	-	112
j	-	108
i	12	104

3)

```
ip = &i ;
```

ip	104	112
j	-	108
i	12	104

4)

```
j = *ip ;
```

```
*ip = 21 ;
```

ip	104	112
j	12	108
i	21	104

Ponteiros como Parâmetro de Função

- Como uma função pode alterar variáveis de quem a chamou?
 - 1) função chamadora passa os endereços dos valores que devem ser modificados ;
 - 2) função chamada deve declarar os endereços recebidos como ponteiros;

```
#include <stdio.h>

void somaprod(int a, int b, int* p, int* q)
{
    *p = a + b ;
    *q = a * b ;
}

int main ()
{
    int s , p ;
    somaprod (3 , 5 , &s , &p) ;
    printf("Soma= %d e Produto = %d \n", s, p);
    return 0 ;
}
```

Passagem de
parâmetros por
referência

Para que Ponteiros são Usados?

- Possibilitar que funções modifiquem os argumentos que recebem;
- Manipular vetores e strings - Vetores são passados como parâmetro através de um ponteiro para o primeiro elemento;
- Criar estruturas de dados mais complexas, como listas encadeadas, árvores binárias etc (não abordado no curso);
- Reduz a necessidade de variáveis globais, melhorando a modularidade;
- Podem ser utilizados para produzir código com melhor desempenho evitando cópias de dados de uma variável para outra.

Operações com Ponteiros

```
#include <stdio.h>

int main( )
{
    int x=5, y=6;
    int *px, *py;
    px = &x;
    py = &y;
    if (px < py)
        printf("py-px = %d\n", py-px);
    else
        printf("px-py = %d\n", px-py);
    return 0;
}
```

Se px e py apontam para
65488 e 65484,
respectivamente...

A saída será:
 $px - py = 1$
(um inteiro de diferença)

- Testes relacionais $\geq$, $\leq$, $<$, $>$ e $==$ são aceitos em ponteiros;
- A diferença entre dois ponteiros será dada na unidade do tipo de dado apontado. Ex.: A diferença entre dois endereços de inteiros será dada em inteiros;

Operações com Ponteiros:

Incremento

```
int main( )
{
    int x=5, y=6;
    int *px, *py;
    px = &x;
    py = &y;
    printf("px = %u\n",px);
    printf("py = %u\n",py);
    py++;
    printf("py = %u\n",py);
    py = px+3;
    printf("py = %u\n",py);
}
```

Podemos utilizar operador de incremento com ponteiros

Podemos fazer aritmética de ponteiros

- Todas as operações são realizadas levando-se em consideração o tamanho do tipo apontado pelo ponteiro (No exemplo: `int`, em geral 4 bytes).

Operações com Ponteiros

- O incremento de um ponteiro acarreta na movimentação do mesmo para o próximo valor do **tipo apontado**
 - Ex: Se *px* é um ponteiro para *int* com valor *3000*, depois de executada a instrução *px++*, o valor de *px* será *3004* e não *3001* !!!
- Deslocamento varia de compilador para compilador dependendo do número de bytes adotado para o referido tipo
- Ponteiros podem não apontar para lugar nenhum. Isto ocorre quando o ponteiro aponta para *0* (zero), ou **NULL** (significa nulo);
- **NULL** é uma constante tipo macro definida em algumas bibliotecas da seguinte maneira:

```
#define NULL 0
```

Operações com Ponteiros

- Na aritmética entre ponteiros é válido:
 - somar ou subtrair um inteiro a um ponteiro ($pi \pm int$) ✓
 - incrementar ou decrementar ponteiros ($pi++$, $pi--$) ✓
 - subtrair ponteiros (produz um inteiro) ($pf - pi$) ✓
 - comparar ponteiros ($>$, $>=$, $<$, $<=$, $==$) ✓
- Não é válido:
 - somar ponteiros ($pi + pf$) ✗
 - multiplicar ou dividir ponteiros ($pi * pf$, pi / pf) ✗
 - operar ponteiros com `double` ou `float` ($pi \pm 2.0$) ✗

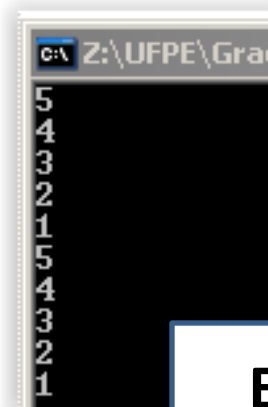
Vetores e Ponteiros

- É possível percorrer vetores utilizando ponteiros:

```
int main( )
{
    int vet[5] = {5, 4, 3, 2, 1};
    int i;
    int *pvet;

    for (i=0; i<5; i++)
        printf("%d\n", vet[i]);

    for (pvet = &vet[0]; pvet<=&vet[4]; pvet++)
        printf("%d\n", *pvet);
}
```



```
C:\Z:\UFPE\Grad
5
4
3
2
1
5
4
3
2
1
```

Equivalente a:
`pvet = vet;`

Aponta para a próxima
posição do vetor

Referencia o conteúdo do
vetor na posição
atualmente apontada

Vetores e Ponteiros

```
int main( )
{
    int vet[5] = {5, 4, 3, 2, 1};
    int i;
    int *pvet;

    for (i=0; i<5; i++)
        printf("%d\n", vet[i]);

    for (pvet = &vet[0]; pvet<=&vet[4]; pvet++)
        printf("%d\n", *pvet);
}
```

pvet



0	...	12	13	14	15	16	17	18	19	20	21	22	23	...
2	1	5				4				3				...

Vetores e Ponteiros

```
int main( )
{
    int vet[5] = {5, 4, 3, 2, 1};
    int i;
    int *pvet;

    for (i=0; i<5; i++)
        printf("%d\n", vet[i]);

    for (pvet = &vet[0]; pvet<=&vet[4]; pvet++)
        printf("%d\n", *pvet);
}
```

pvet



0	...	12	13	14	15	16	17	18	19	20	21	22	23	...
2	1	5				4				3				...

Vetores e Ponteiros

```
int main( )
{
    int vet[5] = {5, 4, 3, 2, 1};
    int i;
    int *pvet;

    for (i=0; i<5; i++)
        printf("%d\n", vet[i]);

    for (pvet = &vet[0]; pvet<=&vet[4]; pvet++)
        printf("%d\n", *pvet);
}
```

pvet



0	...	12	13	14	15	16	17	18	19	20	21	22	23	...
2	1	5				4				3				...

Atividade 1

- Escreva uma função que imprime os elementos de um vetor de float a partir de um endereço inicial (ponteiro) até um endereço final;
- A função recebe como parâmetros dois ponteiros para float (os endereços inicial e final) e deve utilizar notação de ponteiros não de vetores;
- Deve ser criada uma função main() para testar a função implementada com três vetores de tamanhos e conteúdos diferentes;
- A função main deve imprimir duas partes distintas de cada um dos três vetores, utilizando a função criada.

Atividade 2

- Escreva uma função que encontra um valor em um vetor de inteiros e retorna um ponteiro para o primeiro endereço onde este valor foi encontrado ou NULL caso o valor não esteja no vetor;
- A função recebe como parâmetro dois ponteiros para inteiros (os endereços inicial e final) e deve utilizar notação de ponteiros não de vetores;
- Deve ser criada uma função main() para testar a função implementada. Na função main deve ser declarado um vetor de tamanho 10, com três ocorrências do valor 2;
- A main() deve utilizar a função criada para encontrar e imprimir os endereços de memória de todas as ocorrências do valor 2, em um laço, até que todo o vetor tenha sido pesquisado.
- Defina a constante NULL se necessário.